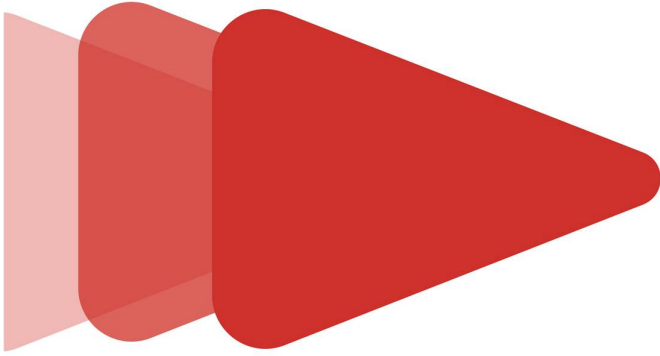




Estándar de Desarrollo Tecnológico Backend-Arquitectura Resiliente

NAC-PTITARE23-00000002

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

Septiembre 2023

Versión: 1.0.0

Firmas de Responsabilidad

Dirección Nacional de Tecnología		Fecha: septiembre 2023	VERSIÓN 1.0.0	PÁGINAS 46
NOMBRE Y CARGO			FIRMA	
APROBADO POR:	Ing. Paulina Correa A. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN-DATOS		 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
ELBORADO POR:	Ing. Christian Arreaga D. EXPERTO NACIONAL ARQUITECTURA SOLUCIONES TECNOLÓGICAS		 Firmado electrónicamente por: CHRISTIAN GUILLERMO ARREAGA DESIDERIO	

Información del Documento

Resumen:	En el presente documento se definen los estándares para la implementación de aplicaciones de microservicios basados en componentes de la plataforma Java EE y el framework Quarkus.
Alcance:	Definición de lineamientos para normar el uso de los diferentes componentes de la plataforma Java EE y el framework Quarkus, también se da lineamientos de temas como: manejo de logs y excepciones, administración de catálogos y parámetros generales. Es menester decir que no se enseña a programar componentes empresariales de la plataforma Java EE ni del framework Quarkus. Detalles adicionales de esta versión se pueden encontrar en la sección Revisiones al Documento.
Audiencia:	Personal de la Dirección Nacional de Tecnología.

Revisiones al Documento

Versión -Fecha	Autor	Descripción
1.0.0 / 2023-09	Christian Arreaga	Emisión Inicial. Esta versión corresponde a una versión preliminar del estándar final que se continuará editando y <u>no</u> incluye las definiciones para: • Aplicaciones de procesamiento por lotes. • Catálogos y parámetros generales • Patrones de diseño • Escalabilidad • Mensajería • Servicios SOAP • Interoperabilidad • Cache de segundo nivel • Pistas de Auditoría • Compilación nativa • Contenerización • Despliegue en ambiente basado en Kubernetes • Ejemplos o arquetipos base De igual manera la versión futura implicará el afinamiento de las reglas dispuestas en esta versión así como también todos los documentos, definiciones, tecnologías archivos de reglas entre otros necesarios para la construcción de lógica de backend.

Enmienda del Documento

El Departamento de Planificación TI y Gestión de Información-Datos tiene la potestad de actualizar el contenido de los Anexos cuando lo considere necesario y no requerirá recolección de firmas para su aprobación. Sin embargo, se comunicarán a las áreas involucradas los cambios realizados, quienes podrán indicar sus observaciones al respecto en los siguientes ocho días laborables, luego de lo cual se considerarán aceptadas, y se procederá a su publicación.

La aplicación de las modificaciones al estándar en proyectos con desarrollos terminados o en curso será consensuado con el departamento de desarrollo, con el fin de evaluar el impacto (positivo o negativo) que tenga sobre el producto o sobre la calidad de este, adicionalmente se debe informar de los acuerdos a las coordinaciones de Control y Aseguramiento de Calidad (QA) y Centro de Cómputo.

Ante cualquier sugerencia de mejora al documento debe ser comunicado al Departamento de Planificación TI y Gestión de Información -Datos para su análisis, evaluación de factibilidad y posterior actualización de la documentación en caso de considerarse necesario.

En caso de ser necesario emitir un lineamiento técnico, aclaración o alcance al presente documento, este podrá ser emitido mediante memorando suscrito por el Jefe Departamental de Planificación TI y Gestión de Información -Datos en coordinación con la Dirección Nacional de Tecnología; y se mantendrá como una disposición vigente hasta su incorporación en la siguiente actualización del documento.

Las modificaciones realizadas deben ser comunicadas al Departamento de Gestión de la Calidad, con la finalidad de llevar seguimiento de las versiones del documento.

Tabla de contenido

1. Introducción	7
1.1. Objetivo	7
2. Estándares Generales	7
3. Pruebas Unitarias	8
3.1. Desarrollo guiado por pruebas (TDD)	8
3.2. Implementación de pruebas unitarias	9
4. Diseño	9
4.1. Generales	9
5. Implementación	10
5.1. Estructura y nomenclatura	10
5.2. Configuración	12
5.3. Configuración de URI (Uniform Resource Identifier)	13
5.4. Validaciones	15
5.5. Respuestas	15
6. Seguridad	16
7. Eventos	17
8. Java Persistence API – JPA	17
8.1. Paginación	18
8.2. Pool de conexiones	18
8.3. Procedimientos almacenados	18
8.4. Entidades	18
8.5. Clases embebible en entidades	19
8.6. Mapeo de relaciones en entidades	20
9. Validaciones de los Java Beans	22
9.1. Constraints de Bean Validation	23
9.2. Validación de métodos en Managed Beans	25
9.3. Creación de constraints personalizados	27
9.4. Personalización de mensajes de error de validación	29
9.5. Validación de servicios web REST	30

9.6.	Mapeo de excepciones de Bean Validation con JAX-RS.....	31
10.	Componentes JEE	32
10.1.	CDI.....	32
10.1.1.	Eventos	33
10.1.2.	Transacciones.....	33
10.1.3.	Scopes.....	34
10.1.4.	Interceptores	34
10.1.5.	Decoradores.....	34
10.1.6.	Estereotipos.....	34
10.1.7.	Métodos producir, disponer y atributos producir en aplicaciones CDI	36
10.1.8.	Message-Driven Bean (MDB)	36
11.	Manejo de logs y excepciones.....	36
11.1.	Manejo de Logs	36
11.1.1.	Implementación general del log	36
11.1.2.	Implementación de mensajes de logs	37
11.1.3.	Implementación del log con MDC	38
11.1.4.	Niveles de un log	38
11.2.	Manejo de Excepciones.....	38
11.2.1.	Implementación de excepciones	38
11.2.2.	Excepciones en el contexto EJB	41
12.	Lógica parametrizable.....	42
13.	Glosario	43
14.	Bibliografía	43
15.	Anexos.....	43
15.1.	Anexo 1: Recursos Relacionados	43
15.2.	Anexo 2: Estándares Relacionados	44
15.3.	Anexo 3: Versiones de tecnologías a utilizar	45

1. Introducción

El Servicio de Rentas Internas para el desarrollo de aplicaciones empresariales, la tecnología estándar es Java con la plataforma empresarial. El presente documento especifica los lineamientos para la construcción de microservicios basados en las tecnologías antes mencionadas y el framework Quarkus.

1.1. Objetivo

Los objetivos del presente documento son:

- Normar la programación de componentes institucionales con la plataforma Java EE.
- Apoyar al control de la calidad de las aplicaciones institucionales.
- Guiar al desarrollador sobre las herramientas a utilizarse, así como también al uso de integración continua.
- Simplificar el uso de tecnologías de la plataforma Java EE.
- Unificar el conocimiento general de la plataforma Java EE para facilitar el desarrollo y mantenimiento de las aplicaciones.
- Establecer lineamientos para el uso de tecnologías Java EE que mejoren el rendimiento, escalabilidad, seguridad y disponibilidad de los componentes Java EE.

Los lineamientos aquí descritos están separados en distintos temas iniciando con temas de estándares generales, y programación Java, que representan los conocimientos básicos para iniciar un sistema institucional.

Los componentes de plataforma Java EE y los patrones de diseño Java EE son los temas centrales de los lineamientos, ya que en estos se especifican temas propios de la arquitectura a utilizarse.

Finalmente se encuentran temas complementarios de los sistemas institucionales los cuales se refieren al manejo de logs, excepciones, pruebas unitarias, y generación de servicios web.

El presente documento entra en vigor a partir de la fecha de aprobación, y debe ser aplicado en la construcción de soluciones de software que usen la plataforma Java EE 7.

Este documento y los documentos denominados guías se encuentran publicados en Alfresco en la ruta: Direcciones Nacionales/Dirección Nacional de Desarrollo Tecnológico/Departamento de Planificación TI/Arquitectura Estándares y Guías.

2. Estándares Generales

Regla: Los estándares definidos en este documento son de cumplimiento obligatorio en el desarrollo de aplicativos institucionales. Para casos de excepción no descritos en este documento se deberá consultar y gestionar la aprobación al Departamento de Planificación TI y Gestión de Información-Datos **previa cualquier implementación**. Para solicitar una excepción utilizar el “Formato solicitud excepciones a estándares” descrito en el [Anexo 1: Recursos Relacionados](#).

Regla: Se debe cumplir con las reglas descritas en los documentos del [Anexo 2: Estándares Relacionados](#) según sea el caso.

Regla: Las versiones de las tecnologías que se pueden usar para la implementación de los proyectos institucionales son las descritas en el [Anexo 3: Versiones de tecnologías a utilizar](#).

Regla: Excluir de las validaciones de PMD y CheckStyle las clases generadas por herramientas de terceros como, por ejemplo:

- ✓ Clases metamodelo basadas en la especificación JPA que tienen el mismo nombre de la clase administrada en la unidad de persistencia más un underscore “_” al final de su nombre, por ejemplo, la clase de persistencia Autor.java su metamodelo es Autor_.java. Del mismo modo se deben excluir de dicha validación los siguientes archivos creados en tiempo de compilación:

```
**/*_.java  
**/*_$bundle.java  
**/*_$logger.java
```

- ✓ Clientes de servicios SOAP generados a partir de la herramienta wsimport incluida en el JDK.

Regla: El uso de librerías de terceros por fuera de las propias del estándar JEE y de aquellas que se encuentran por defecto en los módulos del servidor de aplicaciones necesita del análisis y aprobación del arquitecto TI asignado.

Regla: Es obligatorio el uso de PMD con las reglas personalizadas del SRI, Ver ruta en [Anexo 1: Recursos Relacionados](#).

Utilidad: PMD ayuda a detectar código ineficiente o malos hábitos de programación.

Regla: Es obligatorio la aplicación del Checkstyle definido con los lineamientos por el SRI, Ver versión en [Anexo 1: Recursos Relacionados](#).

Regla: Referente a los catálogos y parámetros generales se deben cumplir las siguientes reglas:

- La estrategia que se debe utilizar para el registro, acceso, actualización y desactivación de parámetros, es a nivel de base de datos.
- Todo catálogo debe estar centralizado en el módulo de ADM (Administración). Para acceder a dicha información se debe utilizar servicios genéricos ofrecidos por dicho módulo.
- Para el ambiente de Contingencia se utilizarán los parámetros correspondientes a este ambiente, configurados en la infraestructura tecnológica.
- Los registros de los parámetros se deben cargar en cache por demanda.

3. Pruebas Unitarias

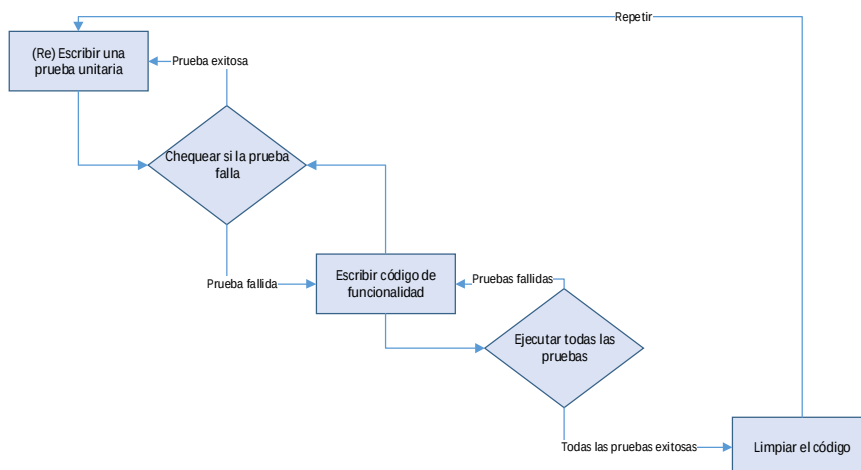
3.1. Desarrollo guiado por pruebas (TDD)

Regla: Para la implementación de un nuevo proyecto se debe aplicar la técnica de desarrollo guiado por pruebas, en inglés Test Driven Development (TDD).

Utilidad: Con el desarrollo guiado por pruebas se favorece la construcción de aplicaciones de mayor calidad; se enfoca a las necesidades del cliente facilitando el entendimiento del problema, y

disminuyendo el tiempo de depuración de errores.

La siguiente imagen muestra el ciclo de desarrollo de pruebas unitarias con TDD:



Ciclo de desarrollo de TDD

3.2. Implementación de pruebas unitarias

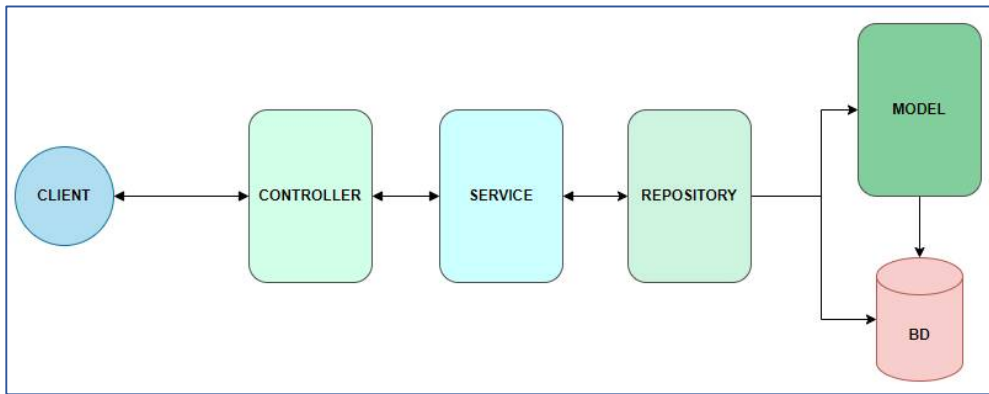
Regla: Para implementar las pruebas unitarias se debe tomar en cuenta los siguientes puntos:

- Los frameworks o librerías que se deben utilizar para hacer las validaciones o afirmaciones de los casos de prueba unitarias es '**JUnit**'.
- Se debe utilizar el framework '**Mockito**' para implementar los casos de prueba unitarias de los servicios de negocio EJBs de la capa de negocio y servicios web de tipo Restful.
- Se deben realizar pruebas unitarias de forma obligatoria de las funcionalidades consideradas como críticas y complejas y de manera opcional de las funcionalidades consideradas como sencillas o no críticas cumpliendo el porcentaje de cobertura.
- No se requiere hacer pruebas unitarias explícitas de: las entidades de persistencia (Entity), java beans (getter y setter), pojos TO (Transfer Object), DAO (Data Access Object), clases usadas para definir excepciones, definición de interfaces, enumeraciones, static Data Model, clases con anotación @Produces, cumpliendo el porcentaje de cobertura.
- El porcentaje de cobertura del código debe respetar lo definido en el Procedimiento de Desarrollo y Mantenimiento de Aplicativos Informáticos Institucionales, sección 3.1.8 "LA CALIDAD" DURANTE EL PROCESO DE DESARROLLO, literal "g".
- Se debe indicar un mensaje descriptivo en todas las validaciones o afirmaciones ('asserts').
- Las pruebas deben ser independientes unas de otras.
- En las pruebas unitarias, no se debe dejar una cadena de caracteres en un fichero de log, es decir no se debe registrar logs de las pruebas para luego ser revisadas su éxito o falla.

4. Diseño

4.1. Generales

Regla: El diseño de la implementación de los microservicios debe seguir la siguiente arquitectura de capas interna.



Componente	Descripción
Client	Representa los clientes que consumirán los servicios.
Controller	Esta capa incluye la capa de controladores o también denominados recursos los cuales son las clases que representan los servicios RESTful por ende aquí se debe definir los endpoints que tendrá nuestro servicio.
Service	Aquí debe colocarse la lógica de negocio que será invocada por la capa de controladores.
Repository	Esta capa contiene las clases de la implementación del patrón Repository para acceso a la capa de datos.
Model	Esta capa alberga las clases que representan a las tablas del modelo de base de datos
BD	Representa la Base de datos

Regla: El código fuente implementado en la capa de controladores no debe contener lógica de negocio y de acceso a base de datos u otros recursos externos tales como otros servicios web, ESB's, entre otros.

Regla: Se debe utilizar la especificación OpenAPI para el diseño de los servicios web RestFul. La versión de esta se puede revisar en [Anexo 3: Versiones de tecnologías a utilizar](#)

5. Implementación

5.1. Estructura y nomenclatura

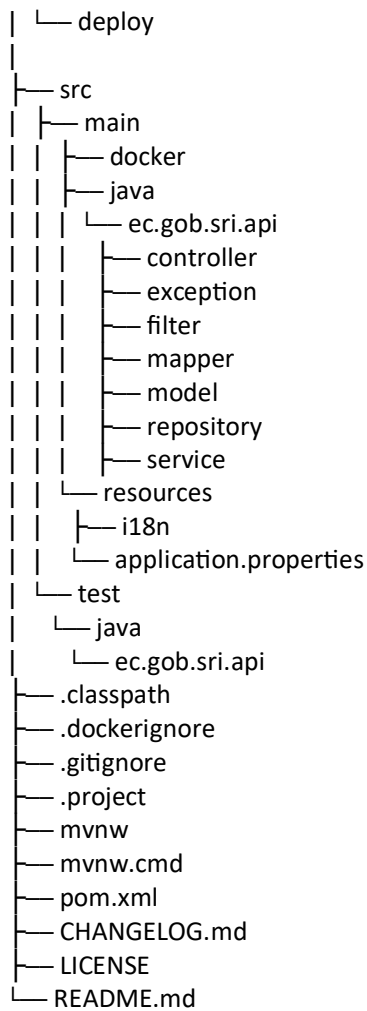
Regla: El nombrado de los proyectos de microservicios basados en Quarkus debe seguir la siguiente estructura:

sri-[dominio del negocio]-service

Regla: La estructura (scaffolding) para los proyectos que se debe seguir es la siguiente:

```

sri-[dominio]-service
├── cicd
└── build
  
```



Regla: La estructura de nombrado de las clases según sea el tipo se debe regir conforme la siguiente tabla y definiciones:

- ✓ El nombrado de las clases debe ser en notación UpperCamelCase.
- ✓ El paquete principal debe ser ec.gob.sri.api.**[dominio de negocio]**.

Componente	Nomenclatura	Ejemplo
Controller	[nombre] Controller Paquete: ec.gob.sri.api.[dominio de negocio].controller	
Exception	[nombre] Exception Paquete: ec.gob.sri.api.[dominio de negocio].exception	
Filter	[nombre] Exception Paquete: ec.gob.sri.api.[dominio de negocio].filter	
Mapper	[nombre] Exception Paquete: ec.gob.sri.api.[dominio de negocio].exception	

Model	El nombre lógico de las entidades conforme al nombre de las tablas en el modelo de base de datos en notación CamelCase.	
Repository	[nombre] Exception Paquete: ec.gob.sri.api.[dominio de negocio].repository	
Service	[nombre] Exception Paquete: ec.gob.sri.api.[dominio de negocio].service	

5.2. Configuración

Regla: La implementación de servicios web REST debe realizarse con JAX-RS (Ver versión en [Anexo 3: Versiones de tecnologías a utilizar](#)).

Regla: La URI base para la aplicación es “**rest**” y se debe establecer en el archivo application.properties en el atributo **quarkus.http.root-path**.

Regla: El formato por defecto para intercambiar información mediante los servicios web es por defecto JSON.

Regla: Para implementaciones de proveedores, mapeos, serialización, deserialización y transformaciones de datos se debe considerar como primera instancia el uso del API Jackson.

Utilidad: Jackson es una biblioteca de Java de usos múltiples para procesar formato de datos JSON y XML disponible para la plataforma StAX, JAXB, etc. Se integra con algunas implementaciones de JAX-RS (Jersey, Apache CXF, RESTEasy, Restlet) y el marco de trabajo de Spring.

Regla: Los servicios REST deben implementarse con sus respectivos métodos de petición HTTP, alineándose a los principios de idempotencia según las siguientes directrices:

Método HTTP	Idempotente	Lineamiento
GET	Si	Utilizar para capacidades de lectura y obtención de datos. Este método no puede cambiar datos. No exponga operaciones inseguras a través de este método.
PUT	Si	Utilizar para capacidades de actualización.
DELETE	Si	Utilizar para capacidades de eliminación. Un borrado lógico, aunque representa una actualización de estados deberá desarrollarse con el verbo: “DELETE” ya que realiza una eliminación de recursos identificado por un URI.
POST	No	Utilizar para capacidades que crean nuevos recursos.
HEAD	Si	Utilizar este método para lectura y obtención de metadatos de los encabezados de respuesta. No exponga operaciones inseguras a través de este

		método.
OPTION	Si	Utilizar este método para comprobar la funcionalidad de un servidor web mediante petición en lugar de un recurso específico. No exponga operaciones inseguras a través de este método.
TRACE	Si	Utilizar este método para fines de depuración y diagnóstico. No exponga operaciones inseguras a través de este método.

Regla: Los servicios web que tienen el propósito de guardar o actualizar información solo pueden enviar los datos en el body del request y no directamente en la URL.

Regla: Las configuraciones propias del aplicativo se deben manejar haciendo uso de la MicroProfile Config specification y el archivo de Quarkus Application configuration ubicado en \$HOME_APPLICATION/config/application.properties¹.

5.3. Configuración de URI (Uniform Resource Identifier)

Regla: Un URI REST debe referirse a un recurso como una entidad de negocio y no como una acción. Los nombres de recursos deben ser sustantivos pluralizados y con estilo: “lowerCamelCase”.

Ejemplo:

```
/* Servicio para administrar la información de los sujetos del módulo de Catastro */
http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos
/* Servicio para administrar la información de las unidades administrativas del módulo de ADM */
http://www.sri.gob.ec/sri-administracion-interfaz/rest/unidadesAdministrativas
```

Regla: Los URI deben seguir una estructura jerárquica predecible para mejorar la comprensibilidad y, por lo tanto, la usabilidad. Predecibles en el sentido de que son coherentes y jerárquicos en el sentido de que los datos tienen relaciones de estructura.

```
/* Inserta un nuevo sujeto en el módulo de Catastro */
POST http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos

/* Obtiene un sujeto del módulo de Catastro con ID: 33457 */
GET http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457

/* Actualizar un sujeto del módulo de Catastro con ID: 33457 */
PUT http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457

/* Elimina un sujeto del módulo de Catastro con ID: 33457 */
```

¹ Véase <https://es.quarkus.io/guides/config-reference#application-properties-file>

DELETE <http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457>

/* Obtiene los todos los medios de contacto del sujeto de Catastro con ID: 33457 */

GET <http://www.sri.gob.ec/sri-catastro-interfaz/rest/sujetos/33457/mediosContacto>

Regla: Cuando no exista un concepto de colección o entidad de negocio es aceptable usar un nombre de recurso en singular.

/* Servicio que obtiene una configuración global desde el módulo de Administración */

GET <http://www.sri.gob.ec/sri-administracion-interfaz/rest/configuracion>

Utilidad: Nombres de recursos apropiados proporcionan contexto para una solicitud de servicio, aumentando la comprensión de la API de servicio. Los recursos se ven jerárquicamente a través de sus nombres de URI, ofreciendo a los consumidores una jerarquía amigable y fácil de entender para aprovechar al máximo sus aplicaciones. Cuando los recursos se nombran bien, una API es intuitiva y fácil de usar.

Consideraciones:

- Las API REST están escritas para los consumidores. El nombre y la estructura de los URIs deben transmitir significado. Diseña para sus clientes, no para sus datos.
- Los recursos que se ven jerárquicamente a través de los URIs es un análisis sobre la granularidad del servicio, contexto de negocio y esquema relacional.

Regla: El uso de URIs configuradas con parámetros basados en expresiones regulares deben contar con la aprobación del arquitecto asignado. Para los casos donde esté justificado en su uso estos no pueden figurar al inicio de la estructura de la URI, sino que deben anteponer siempre un segmento de URL estático.

• Ejemplo:

❌	<pre>@GET @Path("/{orderId:[a-z]\\d{2,3}}") public String getOrders2(@PathParam("orderId") String orderId) { return "orderId: " + orderId; }</pre>
✅	<pre>@GET @Path("obtenerOrden/{orderId:[a-z]\\d{2,3}}") public String getOrders2(@PathParam("orderId") String orderId) { return "orderId: " + orderId; }</pre>

5.4. Validaciones

Regla: Los parámetros de los métodos que se implementan como Servicios REST y que requieran de validación o restricciones (constraints) debe usar el API de Bean Validation con su respectivo archivo de propiedades para la personalización de mensajes.

```
/**
 * Obtiene todos los sujetos del módulo de catastro por estado
 */
@GET
public Response obtenerPorEstado(@NotNull(message =
"{sujeto.consulta.estado.requerido}") @QueryParam("estado") String estado) {
```

Utilidad: JAX-RS soporta Bean Validation para verificar los parámetros utilizados en los recursos REST y su uso permite tener un esquema único de validación que se propaga por todas las capas de la aplicación.

Verifique también [Validación de servicios web REST](#) y [Mapeo de excepciones de Bean Validation con JAX-RS](#)

5.5. Respuestas

Regla: Los servicios REST deben retornar un objeto `javax.ws.rs.core.Response`, el mismo que deberá tener incluidos los datos que se desea retornar si los hubiera y el código de retorno acorde a la tabla de códigos de estado respuesta.

Utilidad: El objeto `javax.ws.rs.core.Response` permite agregar metadatos como cabeceras y códigos de respuesta.

Ejemplo:

```
@GET
@Path("/{messageId}")
@Produces(MediaType.APPLICATION_JSON)
public Response getMessage(@PathParam("messageId") long id) {
    Map<String, String> map = new HashMap<>();
    map.put("1", "abc");
    map.put("2", "def");
    map.put("3", "ghi");
    return Response.status(Status.OK).entity(map).build();
}
```

Output: { "1": "abc", "2": "def", "3": "ghi" }

Regla: Los códigos de estado que se debe implementar en las respuestas de los servicios REST son:

Código Estado	de	Descripción	Contexto de Implementación
200		OK	Utilizar para emitir un resultado de éxito general con contenido en el cuerpo de la respuesta.
201		Creado	Utilizar para emitir un resultado de éxito por una creación exitosa.
204		OK Sin Contenido	Utilizar para emitir un resultado de éxito general sin contenido en el cuerpo de la respuesta.
400		Petición Incorrecta	Utilizar para emitir un resultado de error por validaciones o restricciones en los parámetros de la petición.
401		No Autorizado	Utilizar para emitir un resultado de error de la petición por que el cliente no está autenticado y el recurso es privado.
403		Prohibido	Utilizar para emitir un resultado de error de la petición por que el cliente no está autorizado para realizar la operación solicitada
404		No Encontrado	Utilizar para emitir un resultado de error de la petición por ser un recurso no establecido o inexistente
500		Error Interno del Servidor	Utilizar para emitir un resultado de error general cuando el servidor lanza una excepción no controlada.
503		Servicio Indisponible	Utilizar para emitir un resultado de error cuando el servicio está congestionado o por tareas de mantenimiento

6. Seguridad

Regla: En el código fuente de las aplicaciones NO se deben fijar los valores de: direcciones IP, nombres de dominio, URLs, puertos, protocolos, nombre jndi, nombre de servidor, correo electrónico, usuarios, claves o cualquier otro valor que sea identificado como potencialmente variable en el tiempo de vida del sistema, todos estos valores deben ser manejados mediante parametrización.

Regla: Todas las aplicaciones deben estar integradas al Single Sign On institucional. Para conocer la ubicación de la guía de integración revise [Anexo 1: Recursos Relacionados](#) .

Regla: Los componentes transaccionales que requieran acceder a la base de datos, deben realizar las conexiones según su ambiente (Internet e Intranet). Es decir, si el componente corresponde a funcionalidad que se ejecuta en la intranet solo podrá conectarse a las bases de datos de ese ambiente (ejemplo: INTRASRI, SRI) y si el componente es de internet solo podrá conectarse a las bases de internet (ejemplo: INTER, INTERSRI). Para casos de excepción a esta regla se deberá contar por cada componente con la aprobación del arquitecto asignado al proyecto y la revisión del Departamento de Seguridad Informática.

Utilidad: Manejar el bajo acoplamiento en las aplicaciones y la independencia entre ambientes.

Regla: En la invocación desde un cliente (navegador) a cualquier URL web tales como servicios web (REST / SOAP), páginas web desarrolladas en cualquier tecnología, servlets u otra lógica a nivel del servidor, no se deben pasar como parámetros cualquier información que se pueda obtener mediante otro mecanismo, como por ejemplo de las cabeceras de la petición http, de información de sesión y/o token o que pudiera ser consultada en los repositorios institucionales en base a los datos anteriores.

Ejemplo:

No se debe enviar como parámetro los datos de:

- IPs: en su lugar se debe obtener del request.
- Número de identificación, nombre o correo del usuario: en su lugar se debe consultar en la base de datos con la identificación de la sesión.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Regla: En el caso de que se requiera un servicio web para obtener información que constituyen datos privados (que le pertenece al exclusivamente usuario autenticado), se debe validar que las consultas retornen únicamente datos de ese usuario.

Utilidad: Reducir riesgos de seguridad como, por ejemplo, que un contribuyente pueda obtener información de otro.

Ejemplo: Cuando un usuario ingresa al sistema y consulta una de sus declaraciones, se debe asegurar que dicha consulta no solo se realice la búsqueda en base al código de la declaración, sino que también filtre por la identificación de la persona que está autenticada en sesión.

Regla: La autorización de los endpoints debe estar basado en roles de usuario (autorización basada en roles), los cuales deberán ser parametrizables usando un aplicativo para este propósito.

Regla: El consumo de los servicios debe realizarse a través del protocolo seguro HTTPS.

7. Eventos

8. Java Persistence API – JPA

Regla: Para el acceso a datos se debe utilizar el ORM Hibernate Reactive With Panache provisto por la extensión `io.quarkus:quarkus-hibernate-reactive-panache`, esto para: mapear estructuras de base de datos, ejecutar cualquier sentencia DML (Data Manipulation Language) o ejecución de algún procedimiento almacenado o paquete. Solo se debe utilizar JDBC en sentencias DML, procedimientos o paquetes que tengan que ver con gran cantidad de datos y las mismas estén dando problemas de rendimiento; el Arquitecto TI asignado debe validar la factibilidad de uso de este mecanismo y garantizar la reutilización de componentes genéricos.

Regla: No se deberá utilizar directamente clases de las implementaciones específicas de JPA, en el caso de Jboss Enterprise Application Server las del paquete `org.hibernate.*`. Para los casos de excepción se requiere la aprobación previa del Arquitecto TI asignado.

Regla: Para las consultas que devuelven uno o varios registros se debe utilizar el método `getResultList()`. Si se necesitan realizar validaciones dependiendo del número de registros retornados

se deben realizar en la capa de servicios exceptuando los casos en que dichas validaciones produzcan un cambio de flujo de ejecución para realizar operaciones en la misma capa (DAO).

8.1. Paginación

Regla: Las consultas deben considerar la paginación en bloques de 10,20,30,40 o 50 registros, de ser necesario bloques de mayor magnitud se requiere el análisis y aprobación previa del Arquitecto TI asignado.

8.2. Pool de conexiones

Regla: El tamaño del pool de conexiones por defecto que se debe configurar a través de la propiedad `quarkus.datasource.jdbc.max-size` debe ser de 10.

Regla: Las consultas nativas o NativeQuery no están permitidas.

Regla: Si se requiere utilizar en las consultas los operadores como LIKE, OR, operadores de agregación, y junturas de más de cuatro tablas. que reducen el rendimiento, es necesario validar previamente con el Arquitecto TI asignado para determinar su viabilidad o alternativas de uso.

8.3. Procedimientos almacenados

Regla: La invocación de procedimientos almacenados desde JPA, requiere la revisión y aprobación del Arquitecto TI asignado.

Regla: Para los escenarios aprobados de ejecución de procedimientos almacenados estos no deben contener lógica compleja y su tiempo de procesamiento debe ser menor o igual a 3 minutos.

8.4. Entidades

Regla: Para el desarrollo de las clases de entidad, se deben seguir los siguientes lineamientos:

1. Las anotaciones JPA deben ser puestas sobre los atributos los mismos que hacen referencia a los campos de la tabla mapeada.
2. Todos los tipos de datos de los atributos deben ser tipo objeto, es decir, no utilizar los tipos primitivos java (int, double, boolean, etc.). Así mismo los atributos que corresponden a números decimales deben ser del tipo `BigDecimal`, y los atributos que corresponden números enteros serán del tipo `Long`.
3. Cada entidad debe tener id o clave primaria (primary key) y para denotar a un atributo o campo como clave primaria se debe usar la anotación `@Id` (paquete: `javax.persistence.Id`). El nombre del atributo de la clave primaria debe ser '**codigo**' y tipo de dato '**Long**'.
4. No se permite el uso de claves primarias compuestas (primary key), a excepción de mapeos a aplicaciones legadas donde se tengan claves primarias compuestas.
5. Para la generación de los ids o claves primarias se debe usar la estrategia de secuencias '**SEQUENCE**' (**strategy** = `GenerationType.SEQUENCE`) mediante la creación de una secuencia de base de datos. En la anotación `@SequenceGenerator` se debe poner el atributo '**allocationSize**' con valor '1'.

8.5. Clases embeddable en entidades

Regla: Se debe utilizar clases 'embeddables' (insertables) cuando:

- Más de una entidad utilice esa clase 'embeddable'.
- Por ampliación del dominio de la entidad se requiere agregar nuevos atributos donde por granularidad fina en el modelado del diseño se puede utilizar una clase 'embeddable' para estos nuevos atributos.
- El diseño relacional no se ajusta al modelado orientado a objetos (por ejemplo, una vista que involucra varias tablas se puede representar en varios objetos).

Regla: No se deben utilizar clases 'embeddables' para solucionar problemas de PMD (regla TooManyFields) en entidades.

Regla: El tipo de acceso de las clases 'Embeddable' debe ser '@Access (AccessType.PROPERTY)' a nivel de clase, y para embeberla o utilizarla dentro de una entidad debe ser con la anotación '@Embedded' (paquete javax.persistence.Embedded).

Utilidad: Las clases embeddable (insertable) se utilizan para representar el estado de una entidad, pero son objetos que no tienen una entidad persistente, por lo que para ser persistidos deben ser primero insertados dentro de una entidad. Se debe definir un tipo insertable con la anotación '@Embeddable' (paquete javax.persistence.Embeddable).

Ejemplo:

Se tienen las Entidades JPA de nombres 'Order' y 'User', las cuales tienen dos campos iguales o similares referentes a la dirección () y no se quiere duplicar los mismos campos en las Entidades. Para solucionar esto se debe crear una clase Embeddable (insertable) de nombre 'Address' y luego sea insertada en las entidades mencionadas. En el ejemplo, para crear la clase Embeddable, se debe tener en cuenta que para la entidad 'Order' los nombres de las columnas referentes a la dirección son: **address_street** y **codePostal** y para la entidad 'User' los nombres de las columnas referentes a la dirección son: **user_street** y **user_postal**:

Entidad Order	Entidad User
<pre> @Entity @Table(name = "ORDER") public class Order implements Serializable { @Id @Column(name = "CODIGO_ORDER") private Long id; public Long getId() { return id; } public void setId(Long id) { this.id = id; } @Embedded private Address address; public Address getAddress() { return address; } } public void setAddress() { this.address = address; } </pre>	<pre> @Entity public class User implements Serializable{ @Id @Column(name = "CODIGO_USER") private Long id; public Long getId() { return id; } public void setId(Long id) { this.id = id; } @Embedded @AttributeOverrides({ @AttributeOverride(name="street", column = @Column(name="user_street")), @AttributeOverride(name="codePostal ", column = @Column(name="user_postal")) }) </pre>

}	<pre>private Address address; public Address getAddress() { return address; } public void setAddress() { this.address = address; } }</pre>
Entidad Address (Embedded)	
<pre>@Embedded @Access(AccessType.PROPERTY) Private class Address{ @Column (name="address_street") Private String street; private int codePostal; } public String getStreet() { return street; } public void setStreet() { this.street = street; } public String getCodePostal() { return codePostal; } public void setCodePostal() { this.codePostal = codePostal; } }</pre>	

8.6. Mapeo de relaciones en entidades

Para que se pueda hacer el mapeo de relaciones entre entidades correctamente, la multiplicidad (one-to-one, many-to-one, one-to-many, many-to-many) y tipo de dirección (bidireccional y unidireccional) entre entidades de la unidad de persistencia, el modelado de base de datos debe estar bien definido.

Relaciones bidireccionales

Una relación bidireccional tiene tanto un lado propietario (padre) y un lado inverso (hijo), las mismas en su mapeo deben seguir las siguientes reglas:

6. El lado del hijo debe referirse a su lado padre usando el elemento '**mappedBy**' de las anotaciones @OneToOne, @OneToMany, o @ManyToMany. El elemento mappedBy designa la propiedad o campo en la entidad que es titular o padre de la relación.
7. En el lado de '**many**' en la relación bidireccional 'many-to-one' no se debe definir el elemento 'mappedBy'. El lado 'many' es siempre el lado padre de la relación.
8. Para la relación bidireccional '**one-to-one**', el lado padre debe corresponder al lado que contiene el correspondiente '**foreign key**'.
9. En la entidad padre se debe definir el nombre de la columna que contendrá la clave foránea mediante la anotación '@JoinColumn'.
10. Para la relación bidireccional '**many-to-many**', cualquiera de los dos lados de la relación puede ser el lado propietario o padre. Antes de usar este tipo de relación en el mapeo de entidades, se debe validar con el Arquitecto TI asignado la factibilidad de uso.

Relaciones unidireccionales

En una relación unidireccional, solo una entidad debe tener un campo o propiedad de relación que se refiere a la otra entidad.

Lectura o recuperación de data de entidades

Para la recuperación de datos de las entidades de la unidad de persistencia, se deben seguir las siguientes reglas:

1. Para las relaciones entre entidades '**one-to-one**' (@OneToOne) y '**many-to-one**' (@ManyToOne) se debe utilizar el tipo de lectura **LAZY**.

Ejemplo:

```
@ManyToOne(fetch=FetchType.LAZY)

public VendorPart getVendorPart() {

    return vendorPart

}
```

Para las relaciones entre entidades '**one-to-many**' (@OneToMany) y '**many-to-many**' (@ManyToMany) se debe utilizar el tipo de lectura **LAZY** (valor por defecto).

Se debe usar la cláusula '**JOIN FETCH**' de JPQL cuando se requiera cargar de forma 'EAGER' alguna colección de algún tipo de relación con el tipo de lectura 'LAZY'; hacer esto siempre y cuando el tipo de relación '@OneToOne' o '@OneToMany' no tiene el atributo 'cascade', caso contrario los datos de la colección si están disponibles.

En el ejemplo siguiente se requiere consultar todos los datos de '**LogDenuncia**' y cargar de forma 'EAGER' los datos de la colección de '**DetalleLogDenuncia**':

Entidad Propietario (padre)	Entidad Inverso (hijo)
<pre>@Entity @Table(name = "FFC_LOG_DENUNCIA") public class LogDenuncia implements Serializable { @Id @Column(name "CODIGO_LOG_DENUNCIA") private Long codigo; @OneToMany(mappedBy =logDenuncia") private List<DetalleLogDenuncia> listaDetalleLogDenuncia; } SELECT log FROM LogDenuncia log JOIN FETCH log.listaDetalleLogDenuncia;</pre>	<pre>@Entity @Table(name = "FFC_DETALLE_LOG_DENUNCIA") public class DetalleLogDenuncia implements Serializable { ... @JoinColumn(name ="CODIGO_LOG_DENUNCIA", referencedColumnName="CODIGO_LOG_DEN UNCIA") @ManyToOne(fetchType.LAZY) private LogDenuncia logDenuncia; }</pre>

Para las relaciones entre entidades 'one-to-many' (@OneToMany) y 'many-to-many' (@ManyToMany), se puede establecer a EAGER en alguna relación que se prevea que no pueda afectar al rendimiento de la aplicación y base de datos.

Entidades y operaciones en cascada

En el mapeo de las relaciones entre entidades la opción 'cascade' permite realizar operaciones en cascada sobre las relaciones, para lo cual se debe tomar en cuenta lo siguiente:

1. Utilizar el atributo '**cascade**' solo en relaciones dependientes (padre-hijo) por ejemplo la relación entre Libro y Copia es dependiente, si no existe el libro no tiene sentido que haya copias del mismo. Aplicando la opción *cascade* correcta, permitiría que operaciones sobre el padre se propaguen hacia el hijo.

Se debe utilizar el atributo 'cascade' solo en los tipos de relación 'one-to-one' (@OneToOne) y 'one-to-many' (@OneToMany).

Solo se debe utilizar el tipo de cascada (*CascadeType*) 'PERSIST'.

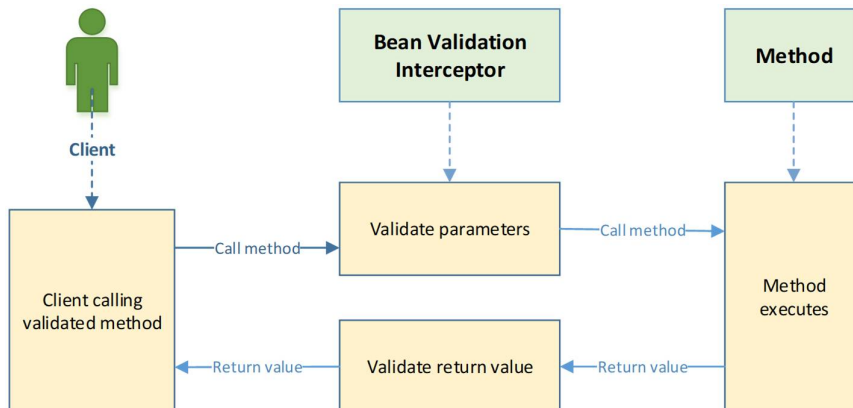
Ejemplo:

Se requieren mapear las relaciones bidireccionales de las entidades llamadas 'LogDenuncia' (hijo) y 'DetalleLogDenuncia' (padre), donde la relación entre las entidades es 'one-to-many' y además se requiere que se utilice el atributo 'cascade' '**PERSIST**'

Entidad Propietario (padre)	Entidad Inverso (hijo)
<pre> @Entity @Table(name = "FFC_LOG_DENUNCIA") public class LogDenuncia implements Serializable { @Id @SequenceGenerator(name = "FFC_SEQ_LOG_DENUNCIA", sequenceName = "FFC_SEQ_LOG_DENUNCIA", allocationSize = 1) @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "FFC_SEQ_LOG_DENUNCIA") @Column(name = "CODIGO_LOG_DENUNCIA") private Long codigo; @OneToMany(cascade = CascadeType.PERSIST, mappedBy = "logDenuncia") private List<DetalleLogDenuncia> listaDetalleLogDenuncia; } </pre>	<pre> @Entity @Table(name = "FFC_DETALLE_LOG_DENUNCIA") public class DetalleLogDenuncia implements Serializable { @JoinColumn(name = "CODIGO_LOG_DENUNCIA", referencedColumnName = "CODIGO_LOG_DENUNCIA") @ManyToOne(fetch=FetchType.LAZY) private LogDenuncia logDenuncia; } </pre>

9. Validaciones de los Java Beans

El API Java para Validación de JavaBeans (Bean Validation) proporciona un mecanismo para la validación de datos de la aplicación. Bean Validation define un modelo de **constraints** (restricciones) basado en anotaciones, las mismas que pueden ser colocadas sobre un campo(field), método o una clase de un componente JavaBeans, como un managed bean o una Entity. La siguiente imagen muestra el ciclo de vida de Bean Validation al ejecutar métodos.



Ciclo de vida de Bean Validation al ejecutar métodos.

Reglas:

2. Para los atributos de una Entidad de persistencia que requieran validación se debe usar el API de Bean Validation. Los atributos a validar no necesariamente pueden ser datos de ingreso o salida de una página web.
3. Para los atributos, o parámetros de métodos (ver sección: *Validación de métodos en Managed Beans*) no estáticos de un componente web *managed bean* que requieran validación se debe usar el API de Bean Validation. Solo validar en el *managed bean* con Bean Validation los campos o propiedades que forman parte de la página web como datos de ingreso.
4. En los parámetros de los métodos de web services tipo REST que requieran validación se debe usar el API de Bean Validation.

Utilidad: El uso del API de Bean Validation permite tener un único esquema de validación de JavaBeans que se propague por todas las capas de la aplicación.

9.1. Constraints de Bean Validation

Bean Validation provee un conjunto de *constraints* (restricciones), las cuales son anotaciones que pueden ser utilizadas en los atributos, o parámetros de métodos de un managed bean o web service tipo REST. Los constraints que se deben utilizar son los siguientes (paquete `javax.validation.constraints`):

Constraint	Descripción	Ejemplo
@DecimalMax	El campo tiene que ser un número cuyo valor sea menor o igual a un máximo especificado.	@DecimalMax (value = "7000.00", message = "{invalid.salaryDecimalMax}") private BigDecimal sueldo;
@DecimalMin	El campo tiene que ser un número cuyo valor sea mayor o igual a un mínimo especificado.	@DecimalMin (value = "550.00", message = "{invalid.salaryDecimalMin}") private BigDecimal sueldo;
	El campo tiene que ser un número cuyo	@Digits (integer=4, fraction=2, message =

@Digits	valor se encuentre en un rango definido. El elemento integer especifica el máximo de dígitos enteros para el número, y el elemento fracción especifica el máximo de dígitos fraccionarios para el número	"{invalid.discountNumberRange}") private BigDecimal descuento;
@Future	El campo tiene que ser una fecha en un futuro.	@Future (message = "{invalid.dateFuture}") private Date eventDate;
@Max	El campo tiene que ser un número cuyo valor sea menor o igual a un máximo especificado.	@Max (value = 120, message = "{invalid.ageMax}") private int edad;
@Min	El campo tiene que ser un número cuyo valor sea mayor o igual a un mínimo especificado.	@Min (value = 18, message = "{invalid.ageMin}") private int edad;
@NotNull	El campo no puede tener valor null.	@NotNull (message = "{invalid.firstName}") private String firstName;
@Past	El campo tiene que ser una fecha en el pasado.	@Temporal(javax.persistence.TemporalType.DATE) @Past (message = "{invalid.datepast}") private Date birthday;
@Pattern	La cadena tiene que coincidir con el patrón de expresión regular especificado.	@Pattern (regexp = "[a-z0-9!#\$%&'*/+=?^_`{ }~-]+(?:\\. + "[a-z0-9!#\$%&'*/+=?^_`{ }~-]+)*@" + "(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\\.)+[a-z0-9]" + "(?:[a-z0-9-]*[a-z0-9])?", message = "{invalid.email}") private String email;
@Size	El tamaño del campo tiene que estar dentro de un límite especificado (incluyente).	@Size (min=2, max=10, message = "{invalid.sizeName}") private String firstName; @Size (min=2, message = "{invalid.sizeLastName}") private String lastName;
@Valid	Valida un objeto java bean que esta como parámetro en: un método de un componente web managed bean o en un recurso Restful.	public String metodo(@NotNull @Valid Contacto contacto){...}

Regla: Todos los campos o propiedades que tengan los constraints (@Size, @Digits, @DecimalMin,

@DecimalMax, @Valid) debe ir precedido por el constraint @NotNull.

Ejemplos:

```
@NotNull(message = "{invalid.firsName}")
@Size(min=2, max=10, message = "{invalid.sizeName}")
private String firstName;

@NotNull(message = "{invalid.notNullTotalSalary}")
@Digits(integer=4, fraction=2, message = "{invalid.totalSalaryNumberRange}")
private BigDecimal totalSueldo;

@POST
@Path("/ingresar")
@Produces(MediaType.APPLICATION_JSON)
public String ingresar(@NotNull(message = "{contact.object.notNull}") @Valid Contacto
contacto){
    return "EXITO";
}
```

Importante:

Si dentro de una entidad de persistencia se pone en algún campo (field) o atributo validaciones de la anotación javax.persistence.**@Column** conjuntamente con un constraint de Bean Validation, la validación que tiene prioridad en ejecutarse es la de Bean Validation. Esto debido a que la validación de @Column que es de JPA se ejecuta en la capa de persistencia, mientras que los constraint de Bean Validation se ejecutan en la capa de presentación.

9.2. Validación de métodos en Managed Beans

Los constraints de Bean Validation deben ser colocados en los parámetros de los métodos no estáticos que requieran ser validados antes de ejecutar la lógica de negocio, para lo cual tomar en cuenta las siguientes reglas:

5. Deben llevar constraints de Bean Validation solo los parámetros de métodos de los componentes web Managed Beans que requieran validación transversal a la lógica de negocio, es decir no validar con Bean Validation los parámetros que NO tengan que ver nada con la lógica de negocio.
6. Deben llevar constraints de Bean Validation solo los parámetros de métodos de los componentes web Managed Beans, y que además el método va a ser invocado solo entre managed beans. En el siguiente ejemplo, si el método *calcularTotalSueldo()* es invocado desde el propio managed bean *SalaryController* los constraints (@NotNull y @Digits) no serán tomados en cuenta:

```
@Named

@RequestScoped
```

```

public class SalaryController implements Serializable {

    @Inject ContactController contactoController;

    public SalaryController() { }

    public BigDecimal calcularTotalSueldo(@NotNull @Digits(integer=3, fraction=2, message = "{invalid.reserveFundsNumberRange}") BigDecimal fondosReserva){

        BigDecimal                                totalSueldo
        =contactoController.getSueldo().subtract(contactoController.getDescuento());

        totalSueldo = totalSueldo.add(fondosReserva);

        return totalSueldo;

    }

```

No se deben usar constraints de Bean Validation en los métodos que se está sobre escribiendo de una clase heredada en un managed bean, como el siguiente ejemplo erróneo:

```

public class Persona {

    ...

    public void setEmail(String email) { ... }

}

public class Empleado extends Persona {

    ...

    @Override

    public void setEmail(@NotNull String email) { ... }

}

```

Cuando se invoque un método de un componente web managed bean que contenga en los parámetros de entrada constraints de Bean Validation, se debe capturar `javax.validation.ConstraintViolationException` poniendo en el log solo el mensaje de la excepción, de esta manera ayudará a identificar en que parámetro se está violando el constraint y se debe retornar al usuario final un mensaje personalizado entendible.

Ejemplo:

```

@Named

@SessionScoped

```

```

public class ContactController implements Serializable {

    @Inject SalaryController salaryController; //componente web Managed Bean

    public String create() {

        try {

            BigDecimal totalSueldo = salaryController.calcularTotalSueldo(fondosReserva);

            current.setTotalSueldo(totalSueldo);

            getFacade().create(current);

            return "";

        } catch (ConstraintViolationException e) {

            logger.error(e.getMessage());

            //Mensaje que retorna al usuario final

            Util.addErrorMessage(e,
            ResourceBundle.getBundle("/messages").getString("PersistenceErrorOccured"));

            return "";

        } catch (Exception e) {

            logger.error(e);

            return "";

        }

    }

}

```

9.3. Creación de constraints personalizados

Regla: Cuando exista el requerimiento de validar (validación personalizada) algún campo (field) con el API de Bean Validation, y que con los constraints dados por el API no es posible hacer tal validación, entonces se debe hacer un constraint personalizado.

Utilidad: La creación y uso de constraints personalizados permite esconder cierta validación para un campo, en donde el uso de esa validación se reduce en añadir una simple anotación en el campo que requiera ese constraint personalizado.

Ejemplo:

Se tiene un campo (field) tipo 'String' llamado '**email**' que forma parte del java bean llamado

'**Contacto**', para lo cual se requiere hacer un constraint personalizado para validar el valor del correo electrónico del campo 'email'. Para implementar el constraint personalizado se debe hacer lo siguiente:

Crear un cualificador o anotación personalizada que representa el constraint personalizado, para este caso se crea la interface '**Email**' anotada con `javax.validation.@Constraint`.

```
@Documented

@Target({ ElementType.ANNOTATION_TYPE,

ElementType.METHOD,

ElementType.FIELD,

ElementType.PARAMETER })

@Retention(RetentionPolicy.RUNTIME)

@Constraint(validatedBy = EmailValidator.class)

public @interface Email {

String message() default "{invalid.email}";

Class<?>[] groups() default {};

Class<? extends Payload>[] payload() default {};

}
```

Implementar la interface '`javax.validation.ConstraintValidator`', en cuya implementación se pone la lógica de la validación personalizada del constraint.

```
public class EmailValidator implements ConstraintValidator<Email, String> {

private static final String PATTERN_EMAIL= "[a-z0-9!#$%&'*/+=?^_`{|}~-]+(?:\\.[a-z0-9!#$%&'*/+=?^_`{|}~-]+)*@(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?";

public void initialize(Email constraintAnnotation) {

System.out.println("EmailValidator.initialize");

}

public boolean isValid(String value, ConstraintValidatorContext context) {
```

```

Pattern patron = Pattern.compile(PATTERN_EMAIL);

Matcher m = patron.matcher(value);

if (m.find()) {
    return true;
}

return false;
}
}

```

Finalmente, para utilizar al constraint personalizado, solo se debe usar el nombre de la anotación creada anteriormente en el campo 'email', ejemplo:

```

@Email
private String email;

```

Para detalles del código referente a ejemplos de uso sobre este tema (constraints personalizados), ir al proyecto ejemplos-estandar-Java EE7 ([Ver Anexo 2: Listado de ejemplos](#)).

9.4. Personalización de mensajes de error de validación

Bean Validation incluye un recurso para mensajes por defecto correspondiente a los constraints.

Regla: Todos los mensajes de los constraints de Bean Validation que se utilicen en la aplicación, deben tener un mensaje descriptivo personalizado. Para esto se debe crear el archivo de propiedades llamado **messages_es.properties** y ubicarlo en la aplicación en la parte correspondiente a la capa de presentación en la ruta **/src/main/resources**, en este archivo se deben poner los mensajes como clave-valor, ejemplo:

```

invalid.email=No es una dirección de correo electrónico válida.

invalid.datepast=Fecha de nacimiento debe ser del pasado.

invalid.lastName=Apellido es obligatorio.

invalid.sizeName=El tamaño del nombre es incorrecto min2, max10.

```

```

@Entity

@Table(name="CONTACTO")

public class Contact implements Serializable {

    @NotNull(message = "{invalid.lastName}")

```

```
@Size(min=2, message = "{invalid.sizeLastName}")
```

```
private String lastName;
```

```
...
```

```
}
```

Importante: El archivo **messages_es.properties** creado, es para localización de mensajes en español indistintamente del país; pero si se requiere crear un archivo de mensajes con localización diferente se debe concatenar el **prefijo del locale del idioma** al nombre del archivo base (MensajesDeValidacion.properties); ejemplo de nombre de archivo de mensajes en idioma inglés indistintamente del país **messages_en.properties**:

```
invalid.email=Correo inválido.
```

```
invalid.datepast=La fecha debe ser anterior.
```

Ejemplo:

Si tiene una Entidad de persistencia llamada 'Contact' y un componente web Managed Bean llamado 'ContactController', en los cuales se requiere validar ciertos campos mediante el API Bean Validation. Además, se tiene un managed bean llamado 'SalaryController' donde existe un método llamado 'calcularTotalSueldo' y que al mismo se requiere añadir constraints de Bean Validation a su parámetro de entrada; implementar lo solicitado.

Para detalles del código referente al ejemplo, ir al proyecto *ejemplos-estandar-Java EE7 (módulo Java EE7-bean-validation – submódulo bean-validation-managed-bean-entity)* ver [Anexo 2: Listado de ejemplos](#).

Regla: Los mensajes de error de usuario final presentados en los sistemas no deben incluir ningún registro de información que pueda comprometer la seguridad de los sistemas y ayudar a un atacante, por ejemplo, no se debe incluir en los errores: identificadores de la sesión, direcciones IP, puertos, nombres de servidores, versiones de software/framework, traza de error, información de identificación personal de un usuario, información reservada.

Regla: Se debe aplicar todos los lineamientos que apliquen de manera general o orientadas a la interacción HTTP descritas en del Estándar de Desarrollo de Aplicaciones Java-Seguridad la ubicación de este se puede ver el [Anexo 2: Estándares Relacionados](#).

9.5. Validación de servicios web REST

Regla: Deben llevar constraints de Bean Validation solo los parámetros de los métodos o recursos de un servicio web REST que requieran ser validados antes de ejecutar la lógica en el recurso.

Utilidad: JAX-RS soporta Bean Validation para validar los recursos REST, con lo cual se puede: añadir constraints a los parámetros de los métodos o recursos y asegurar que un objeto java bean sea validado cuando éste es pasado como parámetro en un recurso.

Ejemplo:

Se tiene el servicio web REST llamado '**Persons**', el cual contiene el recurso llamado '**create**' y se

requiere validar con Bean Validation sus parámetros 'id' y 'name':

```
@Path("persons")

public class Persons {

    @POST
    @Path("create")
    @Produces(MediaType.APPLICATION_JSON)
    public Response createPerson(
        @FormParam("id")
        @NotNull(message = "{person.id.notnull}")
        @Pattern(regexp = "[0-9]+", message = "{person.id.pattern}")
        String id,
        @FormParam("name")
        @Tamaño String name) /* @Tamaño es un constraint de Bean Validation personalizado */ {
        Person person = new Person();
        person.setId(Integer.valueOf(id));
        person.setName(name);
        persons.put(id, person);
        return Response.status(Response.Status.CREATED).entity(person).build();
    }
    ...
}
```

9.6. Mapeo de excepciones de Bean Validation con JAX-RS

Si una `javax.validation.ValidationException` o alguna subclase de `ValidationException` excepto `ConstraintValidationException` es lanzada, el JAX-RS en tiempo de ejecución responderá la petición del cliente con un error **500** (Internal Server Error) HTTP. Por otro lado, si un `ConstraintValidationException` es lanzado, el JAX-RS en tiempo de ejecución responderá la petición del cliente con uno de los siguientes errores: **500** (Internal Server Error) o **400** (Bad Request). Este comportamiento puede ser personalizado con lo siguiente:

Regla: Para enviar como respuesta (*personalizada*) al cliente los mensajes dados por Bean Validation al hacer una petición a algún recurso REST, se debe implementar la interface `javax.ws.rs.ext.ExceptionMapper` que mapea excepciones del tipo `ValidationException`. La implementación de `ExceptionMapper` es lanzada cuando alguna validación con Bean Validation falla.

Ejemplo:

Se requiere implementar la interface `ExceptionMapper` para capturar los errores lanzados por Bean Validation al ejecutar métodos o recursos REST.

```
@javax.ws.rs.ext.Provider
public class ValidationExceptionMapper implements ExceptionMapper<ValidationException>
{

    public Response toResponse(final ValidationException exception) {
        if (exception instanceof ConstraintViolationException) {
```

```

    LOGGER.log(Level.FINER, "Following ConstraintViolations has been encountered.",
exception);
    final ConstraintViolationException cve = (ConstraintViolationException) exception;
    final Response.ResponseBuilder response = Response.status(getStatus(cve));

    final List<Variant> variants =
Variant.mediaTypes(MediaType.APPLICATION_XML_TYPE,MediaType.APPLICATION_JSON_TYPE).build();
    final Variant variant = request.get().selectVariant(variants);
    if (variant != null) {
        response.type(variant.getMediaType());
    }
    response.entity(new
GenericEntity<List<ValidationError>>(getEntity(cve.getConstraintViolations()),new
GenericType<List<ValidationError>>() {}.getType()));

    return response.build();
} else {

    LOGGER.log(Level.WARNING, "Unexpected Bean Validation problem.",exception);
    return Response.serverError().entity(exception.toString()).build();
}
}

```

10.Componentes JEE

10.1. CDI

Regla: La inyección de dependencias se lo debe hacer con la anotación `@Inject`.

Utilidad:

Regla: Para definir el scope se debe por usar por defecto la anotación `@ApplicationScoped`. Si se requiere el uso de otro tipo de scope debe solicitarse la asignación de un arquitecto para realizar una revisión conjunta.

Utilidad:

Regla: Para hacer la inyección de dependencia de los beans (POJOs) implementados con o sin cualificadores, se debe inyectar usando la anotación `@Inject` (paquete `javax.inject.Inject`).

Utilidad: CDI da la facilidad de inyectar POJOs con solo utilizar la anotación `@Inject` y cumplir la especificación Java EE7.

Ejemplo: Ver el proyecto base en [Anexo 1: Recursos Relacionados](#).

Regla: Seguir la guía de referencia para Quarkus la misma se puede encontrar en: [Anexo 1: Recursos Relacionados](#).

10.1.1. Eventos

Regla: Los eventos que pueden ser usados son los siguientes, para cualquier otro tipo de evento solicitar el acompañamiento de un arquitecto.

- ✓ io.quarkus.runtime.ShutdownEvent
- ✓ io.quarkus.runtime.StartupEvent

Utilidad:

10.1.2. Transacciones

Regla: Se debe utilizar el administrador de transacciones de Quarkus para lo cual se debe hacer uso de la anotación `jakarta.transaction.Transactional`. Se pueden utilizar los siguientes atributos de la anotación:

Atributo	Descripción
@Transactional(REQUIRED) (default)	Inicia una transacción si no se inició ninguna; de lo contrario, permanece con la existente
@Transactional(REQUIRES_NEW)	Inicia una transacción si no se inició ninguna; si se inició una existente, lo suspende y comienza una nueva para el límite de ese método.
@Transactional(MANDATORY)	Falla si no se inició ninguna transacción; De lo contrario, funciona dentro de la transacción existente.
@Transactional(SUPPORTS)	Si se inició una transacción, la suspende y trabaja sin transacción dentro del límite del método; de lo contrario funciona sin transacción.
@Transactional(NOT_SUPPORTED)	Si se inició una transacción, se une a ella; de lo contrario funciona sin transacción.
@Transactional(NEVER)	Si se inició una transacción, genera una excepción; de lo contrario funciona sin transacción.

Regla: Se debe usar `@Transactional` sin parámetros por defecto.

Regla: En los casos que sea necesario realizar un control manual o a nivel de programación de la transacción se puede usar las siguientes alternativas:

- `jakarta.transaction.TransactionManager`
- `io.quarkus.narayana.jta.QuarkusTransaction`

Regla: En un método de un enterprise bean que lance una excepción de aplicación (*excepción que ocurre mientras se ejecuta lógica de negocio*) y para marcar a la transacción para que haga rollback, en la captura de la excepción se debe usar el método `setRollbackOnly()` de la interface `jakarta.transaction.TransactionManager`.

Consideración. - Hay varios buenos motivos para que las transacciones funcionen así. Las excepciones chequeadas pueden usarse para identificación de eventos o para redirigir el procesamiento basándose en ciertas condiciones. Pero, además, el código fuente de la aplicación podría tomar medidas correctivas en algunas excepciones chequeadas, y permitir que la transacción se complete. Por ejemplo, en una aplicación bancaria, para completar la transacción de transferencia de fondos se necesita enviar una auditoría como parte del proceso, al inicio y al final de la transacción.

Si la transacción de la transferencia de fondos fallara (errores de aplicación o de infraestructura) se lanzará algún tipo de excepción chequeada indicando que la transacción no pudo ejecutarse.

Si las excepciones chequeadas causaran un rollback automático, toda la transferencia de fondo tendría un rollback. Al no realizar un rollback automático sobre excepciones chequeadas, es posible capturar la excepción y realizar alguna acción correctiva (como enviar la auditoria de la transacción).

10.1.3. Scopes

Regla: En lo que respecta las anotaciones de los estados (scopes) de los beans se debe cumplir lo siguiente:

- Para los beans que necesiten instanciarse una sola vez a nivel de aplicación

Anotación	Descripción
@ApplicationScoped	Se utiliza para definir un bean como un bean con ámbito de aplicación, lo que significa que se creará solo una vez por aplicación y se compartirá entre todos los clientes. Esto puede resultar útil para beans cuya creación es costosa o para beans que deben compartirse entre varios clientes.
@Singleton	A diferencia de @ApplicationScoped, si un bean tiene la anotación @Singleton, se crea una instancia cuando se inyecta en otro bean. Pero es irrelevante si alguna vez se accede al bean.

- Para los beans que necesitan instanciarse por cada solicitud.

Anotación	Descripción
@RequestScoped	Si una clase de bean tiene la anotación @RequestScoped, CDI creará una instancia del bean de forma perezosa durante la primera llamada a un método de bean. Un bean de este tipo vive sólo dentro de una cadena que se utiliza para procesar una única solicitud HTTP.

10.1.4. Interceptores

Regla: Se debe analizar con el Arquitecto TI asignado la viabilidad de utilizar interceptores en el desarrollo de la aplicación.

10.1.5. Decoradores

Regla: Se debe analizar con el Arquitecto TI asignado la viabilidad de utilizar decoradores en el desarrollo de la aplicación, debido a que los decoradores pueden conducir a un sistema a la existencia de una gran cantidad de decoradores similares que pueden ser difíciles de mantener y distinguir.

10.1.6. Estereotipos

Regla: Los estereotipos permiten agrupar conjuntos de anotaciones bajo otra anotación única, por lo tanto, si en la aplicación se tiene más de dos beans que tienen iguales características (combinaciones de mismo: scope, @Named, interceptor binding, u otro cualificador o anotación) se debe utilizar estereotipos creados por el desarrollador, para lo cual se debe utilizar la anotación **@Stereotype** (paquete javax.enterprise.inject.Stereotype).

Utilidad: Ayuda a la reutilización de anotaciones sin tener que escribirlas explícitamente al crear los

beans. Los estereotipos pueden ser útiles en una aplicación en la que se tiene un número de beans que ejecutan iguales funciones.

Ejemplos:

CDI dispone de un estereotipo predefinido llamado **@Model** (paquete `javax.enterprise.inject.Model`) que está destinado a ser utilizado en beans que definen la capa del modelo en una arquitectura MVC. Tal estereotipo especifica que un bean tendría `@Named` y `@RequestScoped`.

Entonces, si un bean requiere ser anotado con `@Name` y `@RequestScoped` se debe usar el estereotipo predefinido llamado `Model` y no crear uno nuevo.

```
@Model
public class MockLoginAction extends LoginAction { ... }
```

Se requiere crear un estereotipo de nombre `Action`, el mismo que tenga el scope `@RequestScoped`, `@Named` y el interceptor binding `@Logged`. Todos los beans anotados con `@Action` tendrían: request scope, el nombrado por default de EL, y el interceptor binding `@Logged`.

```
@Model
@Logged
@Stereotype
@Target(TYPE)
@Retention(RUNTIME)
public @interface Action {}
```

Se requiere crear un estereotipo de nombre `'Simulacion'`, el mismo que tenga la anotación `@Alternative`. Todos los beans anotados con `'@Simulacion'` serían `alternatives`.

```
@Alternative
@Stereotype
@Target(TYPE)
@Retention(RUNTIME)
public @interface Simulacion {}
```

Es posible aplicar varios estereotipos al mismo bean, para este caso al bean llamado `'MockLoginAction'` se anotará con los estereotipos antes creados `@Action` y `@Simulacion`,

```
@Action
@Simulacion
public class MockLoginAction extends LoginAction { ... }
```

Si ya se tiene un estereotipo definido y solo se requiere modificar su scope o ámbito se debe **sobrescribir** el scope especificado en el estereotipo, simplemente especificando un diferente scope para el bean. Por ejemplo, al bean `'MockLoginAction'` se le da el `'session scope'` en vez de `'request scope'` (el que está definido el estereotipo `@Action`).

```
@SessionScoped
@Action
```

```
@Simulacion
public class MockLoginAction extends LoginAction { ... }
```

10.1.7. Métodos producir, disponer y atributos producer en aplicaciones CDI

10.1.8. Message-Driven Bean (MDB)

Regla: Se debe analizar con el Arquitecto TI asignado la viabilidad de utilizar MDB's en el desarrollo de la aplicación.

11. Manejo de logs y excepciones

11.1. Manejo de Logs

11.1.1. Implementación general del log

Regla: Se establece que la implementación de los log's en las aplicaciones Java EE 7 se realice con el framework JBoss Logging.

Utilidad: El framework JBoss Logging presenta una API que desacopla el mecanismo de los logs en la capa media, proporcionando además una manera fácil e innovadora para su implementación con soporte completo enfocado en la internacionalización del mensaje, localización de los logs, etc.

Regla: La implementación de los logs debe ser desarrollada con la siguiente importación:

```
import org.jboss.logging.Logger;
```

Regla: No utilizar: system.out, system.err, printStackTrace. Como mecanismo para el control, depuración y mantenimiento de la aplicación se debe utilizar el registro de log con su nivel respectivo.

Utilidad: Los métodos: system.out, system.err, printStackTrace, son muy costosos y generan problemas de rendimiento y memoria a la aplicación.

Ejemplo:

INCORRECTO:

```
} catch (AuditoriaExcepcion e) {
e.printStackTrace();
return null;
}
```

CORRECTO:

```
} catch (AuditoriaExcepcion e) {
log.error("ERROR AL OBTENER INFORMACION DEL USUARIO AUTENTICADO", e);
return null;
}
```

Regla: Las pruebas unitarias no deben implementar logs.

Consideraciones:

- Implementar el registro de logs solo en casos necesarios.
- El registro de un log no debe causar excepciones ejemplo:

Al ejecutar esta sentencia hay que asegurarse que el objeto politicaAuditoria no sea nulo:

```
log.debug("PROCESANDO POLITICA: " + politicaAuditoria.getCodigoPolitica());
```

Un objeto como usuariosLista, manipulado incorrectamente en el log puede causar excepciones como OutOfMemory, LazyInitializationExceptions, etc:

```
log.debug("PROCESANDO USUARIOS: " + usuariosLista);
```

- La concatenación de Strings es costosa, teniendo en cuenta que en Java los Strings son inmutables y se crea un pool de Strings que se almacena en la memoria dinámica. En la implementación de los log's, se debe definir la creación de Strings en tiempo de ejecución, esto quiere decir que sólo los Strings que se requieran van a ser creados, que implica menos trabajo del Garbage Collector.

Ejemplo:

```
log.debug("Total filas: " + numeroFilas + " del filtro: " + filtroAuxiliar + "");
```

- Asegurarse que el log brinde la información suficiente y consistente.

INCORRECTO:

```
log.debug("INICIO");
```

CORRECTO:

```
log.debug("INICIO DEL PROCESO ASINCRONO DE AUDITORIA: " + politicaAuditoria);
```

Regla: En el control de excepciones el log debe realizarse en lo más alto de la pila y no en cada excepción.

Regla: En los casos que se utilice log en los niveles INFO, TRACE y DEBUG, se deberá verificar que el nivel de log es válido antes de invocarlo, utilizando las funciones: isInfoEnabled, isTraceEnabled, e isDebugEnabled respectivamente.

Utilidad: Comprueba niveles de logs activos antes de registrarlo para evitar problemas de rendimiento y memoria.

Ejemplo:

```
if (log.isDebugEnabled()) {
    log.debug("Error [" + numeroIteracion + "]: ", e);
}
```

11.1.2. Implementación de mensajes de logs

Regla: Todos los mensajes logs deben poseer un código de proyecto general de máximo 9 caracteres según el contexto de negocio y un identificador de rango según su nivel.

Nivel	Identificador	Ejemplo
DEBUG	100	101, 102, ... 199
INFO	200	201, 202, ... 299
WARN	300	301, 302, ... 399
ERROR	400	401, 402, ... 499
FATAL	500	501, 502, ... 599

11.1.3. Implementación del log con MDC

La idea básica del Contexto de Diagnóstico Asignada (MDC) es proporcionar una forma de enriquecer los mensajes de registro con partes de información que podrían no estar disponibles en el ámbito en el que el registro se produce. Por ejemplo:

- En aplicaciones distribuidas o que tienen múltiples clientes simultáneamente
- En una trazabilidad donde se necesite relacionar el log de la aplicación con la pista de auditoría donde el dato de la correlación es el nombre de usuario y/o su dirección IP.

Regla: La implementación de MDC debe estar desacoplada de la lógica de negocio por medio del desarrollo de interceptores. Todas las partes de información que se agregan al log deben ser removidas o limpiadas del contexto de diagnóstico asignado.

11.1.4. Niveles de un log

Regla: A continuación, se definen las reglas para usar niveles de log:

- **DEBUG:** Usar este nivel siempre que se esté investigando un error específico.
- **INFO:** Usar este nivel para emitir información general como: inicio, apagado, versión y configuración de una aplicación. También para emitir información acerca de decisiones importantes en el código o cambios en la configuración.
- **WARN:** Usar este nivel para emitir una advertencia o información que no representan un problema por sí mismo.
- **ERROR:** Usar este nivel para identificar un problema y se tienen que tomar acción sobre esto.
- **FATAL:** Usar este nivel si la aplicación presenta un problema con un estado irrecuperable como por ejemplo excepciones lanzadas por la JVM (OutOfMemoryError), cuando esté relacionado a un apagado del servidor o a matar la sesión.

Regla: El nivel del log que se utiliza en producción es: "ERROR".

Regla: El nivel del log que utilizan en los ambientes no productivos (Pruebas y Certificación) es: "DEBUG".

11.2. Manejo de Excepciones

11.2.1. Implementación de excepciones

Regla: En los modelos y capa de acceso a datos no se debe controlar ni lanzar la excepción. Las excepciones de tipo no controladas que ocurran en la capa de acceso a datos deben ser capturadas en la capa de servicio (beans EJBs), transformadas en una excepción de tipo aplicación y lanzadas a otra capa superior.

Regla: Las excepciones controladas deben ser capturadas en el método en el que se producen (no se consideran clases de modelos y capa de acceso a datos), transformadas en las excepciones de aplicación adecuadas y enviadas hacia las capas superiores.

Regla: Las excepciones de tipo no controlado que se produzcan en la capa de servicio se considerarán en general errores del sistema. Para evitar que su ocurrencia se muestre al usuario se debe utilizar el mecanismo de control de errores web.

Regla: En la capa de presentación o servicios web solo debe realizar el control de las excepciones, en este nivel no deben existir métodos que vuelvan a lanzar las excepciones.

Regla: No se debe capturar la excepción genérica (java.lang.Exception) sino únicamente cuando APIs de terceros lo obligan.

Regla: Evitar que la ocurrencia de excepciones se muestre al usuario final en pantalla de forma incontrolada.

Regla: Mantener un log de todas las excepciones que ocurren en el sistema y que sea lo más legible posible.

Regla: En la capa de vista/controlador se deben capturar tanto las excepciones de tipo aplicación que provienen de las capas inferiores como las que se produzcan en esta capa. Las excepciones serán capturadas en el backing bean dentro de cada uno de los métodos donde se ejecuta el acceso a la capa de servicio.

Regla: El sistema deberá gestionar los mensajes orientados a los usuarios y a los administradores. Los primeros son los que se muestren en las páginas de la aplicación y los segundos los que se registren en el log del sistema.

Regla: Cuando se gestionen recursos externos (archivos, conexiones, etc.) se debe implementar de java.lang.AutoCloseable.

Utilidad: El uso de AutoCloseable hace que el código sea más corto y limpio para el manejo de excepciones.

Ejemplo:

Con múltiples recursos	Autoclosable
<pre> public class CloseableImpl { public void doSomething() throws Exception { // ... } public void close() throws Exception { // ... } public static void main (String[] args) { CloseableImpl impl = new CloseableImpl(); try { impl.doSomething(); } catch (Exception e) { // ex from doSomething } finally { try { // impl.close() must be called explicitly </pre>	<pre> public class AutoCloseableImpl implements AutoCloseable { public void doSomething() throws Exception { // ... } public void close() throws Exception { // ... } public static void main (String[] args) { // impl.close() will be called implicitly try (AutoCloseableImpl impl = new AutoCloseableImpl()) { impl.doSomething(); } catch (Exception e) { // ex from doSomething } </pre>

```

impl.close();
    } catch (Exception e) {
    }
}
}
}
}

```

Regla: Usar catch múltiple, para la agrupación de tipos de Excepciones mediante el signo ‘|’

Ejemplo:

```

try {

    // algún código que lance algunas de las tres excepciones.

} catch (SQLException | IOException e) {

    log.error(e);

} catch (Exception e) {

    log.error(e);

}

```

Regla: Use **Enmascaramiento de Excepciones** como técnica en el manejo de excepciones.

Utilidad:

- Prevenir que lleguen hasta la tope del *call stack* (*pila de llamadas*) todas las posibles excepciones del sistema.
- Se prefiere que los componentes de más alto nivel no conozcan mucho de la implementación de los componentes de más bajo nivel.
- Siempre se debe poner la causa de la excepción en el enmascaramiento.
- Se prefiere el enmascaramiento cuando se requiera agregar información adicional a la excepción.

Ejemplo:

```

try{
    ...
    appService.guardarEntidad(contribuyente);
} catch (SQLException sqlException) {
    throw new AppException("Error guardando la Entidad", sqlException);
}

```

Regla: El manejo de excepciones debe ser “**A prueba de errores**”.

Utilidad: Permite que durante la investigación de errores se tengan los datos correctos, y no se enmascare la excepción original, debido a que la excepción que se lanza siempre es la última excepción.

Ejemplo:

- Manera incorrecta, variable *input* puede ser *null*.

```
InputStream input = null;
```

```

    try {
        InputStream input = new FileInputStream("archivo.txt");
        // hacer algo con el stream
    } catch (IOException e) {
        throw new WrapperException(e);
    } finally {
        try {
            input.close();
        } catch (IOException e) {
            throw new WrapperException(e);
        }
    }
}

```

- Manera correcta, verificación que la variable *input* no sea nula, caso contrario el bloque de código genera un *NullPointerException*.

```

InputStream input = null;

try {
    input = new FileInputStream("archivo.txt");
    // do something with the stream
} catch (IOException e) {
    throw new WrapperException(e);
} finally {
    try {
        if (input != null){
            input.close();
        }
    } catch (IOException e) {
        throw new WrapperException(e);
    }
}

```

Consideración: La última excepción lanzada en un bloque *try/catch/finally* es la excepción que se propaga hacia arriba en la pila de llamadas, por lo tanto, todas las excepciones anteriores desaparecen.

11.2.2. Excepciones en el contexto EJB

Regla: Si una excepción de aplicación no está definida para causar un *rollback* en la transacción, marcar la transacción usando el método *TransactionManager.setRollbackOnly* antes de lanzar la excepción de aplicación.

Utilidad: Marcando la transacción para *rollback* se asegura que la transacción nunca hará *commit*.

Regla: Para el manejo de excepciones del sistema considerar:

- Si el método del *bean ejb* encuentra una excepción o error del sistema, debe simplemente propagar el error desde el método al contenedor (El método no debería capturar la excepción).
- Si el método del *bean ejb* realiza una operación que resulta en una excepción chequeada² en la cual el *bean ejb* no se pueda recuperar, el método del *bean ejb* debe lanzar un *javax.ejb.EJBExcepción* que encapsule la excepción original.
- Cualquier otra condición de error inesperado debería ser reportado usando una *javax.ejb.EJBException*.

Nota:

- La excepción *javax.ejb.EJBException* es *no chequeada*, es una subclase de *java.lang.RuntimeException* y no tiene que estar reportada en la firma del método.

12.Lógica parametrizable

Regla: La lógica de negocio que se prevea pueda cambiar, deberá ser parametrizada mediante uno de los siguientes métodos:

- ✓ Uso de parámetros generales: esta alternativa se aplica en el caso de requerir configuraciones del tipo nombre valor. *Vea también [Catálogos y parámetros generales](#).*

Ejemplo: Parametrización del valor del Impuesto al Valor Agregado.

- ✓ Creación y uso de estructuras de parametrización (tablas propias del sistema): esta alternativa se aplica para funcionalidades de complejidad media por las particularidades de los requerimientos.

Ejemplo: Parametrización de las reglas del Impuesto a la Renta

- ✓ Uso de librería de reglas de negocio (Drools): se utilizará cuando se requiera automatizar decisiones de negocio de complejidad alta que consideren diferentes variables y que pueden ser configuradas como tablas de decisión.

Ejemplo: Notas de crédito para el pago de impuestos que tipo de NC sirve para cada impuesto, para la determinación de la oficina de atención de contribuyente en base a parametros cómo jurisdicción el tipo de trámite, el canal de ingreso.

Utilidad: Permitir la flexibilidad para que los sistemas puedan externalizar la lógica de negocio que se considera parametrizable.

Regla: Para el uso de la librería de reglas de negocio (Drools) se requerirá contar con el análisis y aprobación del Arquitecto TI asignado y se deberá colocar la definición correspondiente en la Arquitectura de la Solución.

Regla: En el caso del uso de la librería de reglas de negocio (Drools), se debe almacenar el archivo de reglas de negocio en la base de datos.

² Una excepción chequeada es aquella que no es subclase de *java.lang.RuntimeException*.

13. Glosario

AJAX.- Asynchronous JavaScript And XML

AOP.- Aspect Oriented Programing

API.- Application Program Interface

BMT.- Bean-Managed Transactions.

CDI.- Contexts and Dependency Injection

DS.-Data source

DML.- Data Manipulation Language

ECI.- Exception Catcher Interceptor

EJB.- Enterprise Java Bean

JAR.- Java Application Resource

JAXB.-Java Architecture for XML Binding

JCP.- Java Community Process

JMS.- Java Messaging Service

JNDI.- Java Naming Directory Interface

MDB.- Message Driven Bean

MDC.- Mapped Diagnostic Context (Contexto de Diagnóstico Asignada)

REST.- La transferencia de estado representacional (en inglés representational state transfer).

WAR.- Web Application Resource

WS.- Web Service

14. Bibliografía

1. Quarkus (2023), Guides - Latest: Tutorials Short and focused exercises to get you going quickly, <https://quarkus.io/guides/> .
2. Eclipse EE4j (2023), The Jakarta EE Tutorial, <https://eclipse-ee4j.github.io/jakartaee-tutorial/>

15. Anexos

15.1. Anexo 1: Recursos Relacionados

Recurso	Ubicación
Formato solicitud excepciones a estándares	
Archivo de reglas PMD	http://sridesa02:8081/repository/releases/ec/gob/sri/resources/reglas-pmd-sri/3.0/reglas-pmd-sri-3.0.xml

Archivo de reglas CheckStyle	http://sridesa02:8081/repository/releases/ec/gob/sri/resources/reglas-checkstyle-sri/2.0/reglas-checkstyle-sri-2.0.xml
Proyecto Base	
Contexts And Dependency Injection Guide	https://es.quarkus.io/guides/cdi-reference

15.2. Anexo 2: Estándares Relacionados

Estándar	Descripción	Ubicación
Interacción con la base de datos	Cumplir los lineamientos establecidos para cada base de datos que existen en la institución, estos lineamientos son de cumplimiento obligatorio y se encuentran descritos en el documento Estándar de Desarrollo – Nombrado de Objetos de Base de Datos , si existiera un caso de excepción a esta norma, debe ser resuelta y acordada con el Departamento de Planificación TI y Gestión de Información -Datos y el Departamento de Infraestructura y Operaciones, estas definiciones deben ser reflejadas en el documento de Arquitectura de la Solución Tecnológica de cada aplicación	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/3bf7bcf9-1b96-43b2-b06e-7e64d0346edd
Desarrollo de aplicaciones Batch	Para la implementación de procesamiento batch se deberá aplicar lo establecido en el documento “Estándar de Desarrollo de Aplicaciones Batch” .	https://sriintra01/share/page/documentdetails?nodeRef=workspace://SpacesStore/3a4db12d-449d-4f9c-929a-d3570a493aaa
Codificación Java Standard Edition	“SRI Convenciones de codificación para Java” .	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/6a5ea3ec-3551-43b3-abb-bcc2674d6488c
SRI Patrones Diseño Java EE	Documento de Patrones de Diseño JEE	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/709fe676-f673-4b03-ba38-d85d449b410c
Estándar de Desarrollo de Aplicaciones Java-Seguridad	Documento de definiciones de seguridad para aplicaciones Java	https://sriintra01/share/page/document-details?nodeRef=workspace://SpacesStore/2f16c3c1-0a29-48b6-a9f2-ec4e737de003

15.3. Anexo 3: Versiones de tecnologías a utilizar

Tecnología	Versión
Java Development Kit (JDK)	Red Hat – OpenJDK 17.x
Java Enterprise Edition	Jakarta EE 10
Apache Maven	3.x
Framework para el desarrollo de microservicios	Quarkus 3.x
Implementación de JAX-RS	Quarkus extension: io.quarkus:quarkus-resteasy-reactive
OpenAPI	3.X